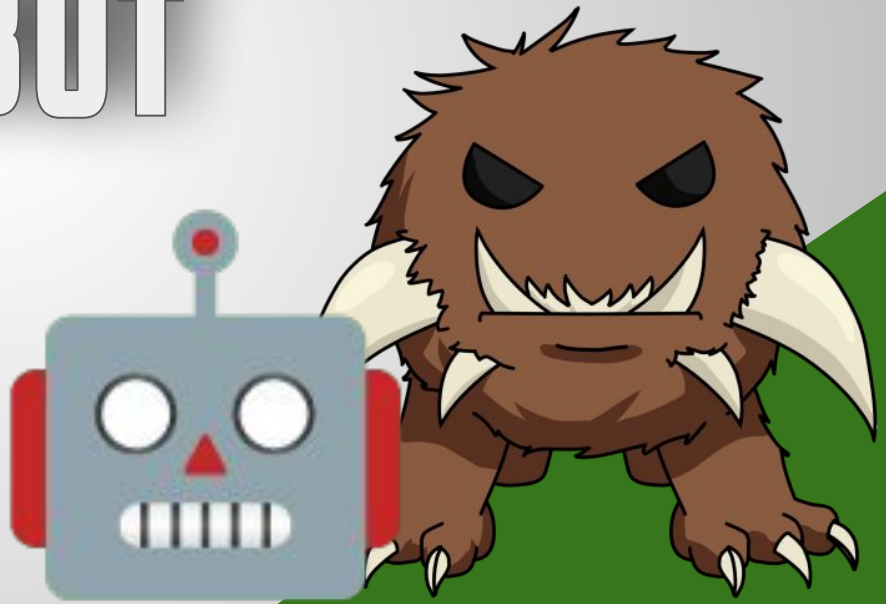
JUEGO DEL WUMPUS Y ROBOT

Inteligencia Artificial

Jimena de Prado
Belisa Yu

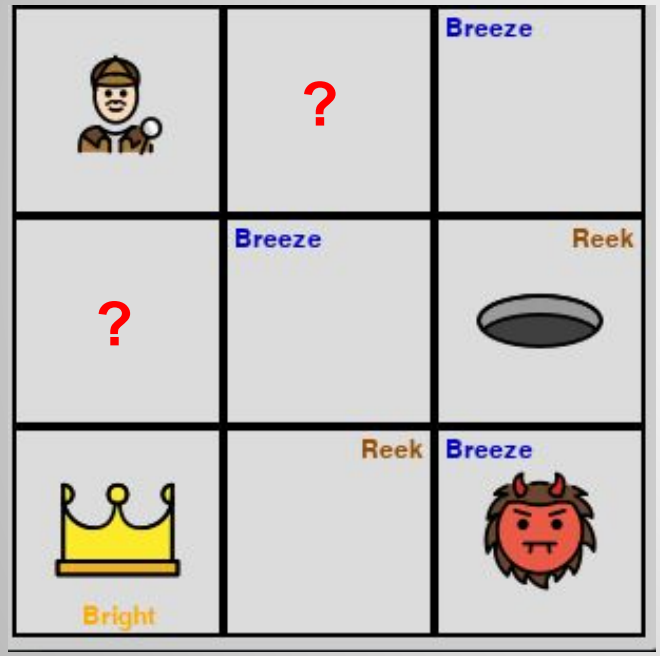


ÍNDICE

1. Resolución SAT y Motor DPLL en el mundo del Wumpus
   a. Modelo lógico del Entorno y Aprendizaje del Agente
   b. Inferencia lógica y optimización algorítmica con DPLL
   c. Resultados y conclusiones
   d. Experimentación

2. Planificación basada en conocimiento: robot de reparto
   a. Problema y formalización con cálculo de situaciones
   b. Axiomas del dominio
   c. Búsqueda hacia delante con BFS
   d. Resultados y conclusiones

Modelo lógico del Entorno y Aprendizaje del Agente



1. **Entorno estocástico**
   a. Tablero aleatorio
   b. Observabilidad parcial

2. **Codificación (id_var & perceive)**
   a. Pistas a números enteros
   b. Hechos: ±Breeze, ±Reek, ±Bright

3. **Formalización lógica -> FNC**
   Well(x,y) · Wumpus(x,y) · Gold(x,y)
   Reek(x,y) <-> Wumpus(vecinos)
   Breeze(x,y) <-> Well(vecinos) → **FNC =** $(\neg B \vee P1 \vee P2) \wedge (\neg P1 \vee B) \wedge (\neg P2 \vee B)$

Inferencia lógica y optimización algorítmica con DPLL

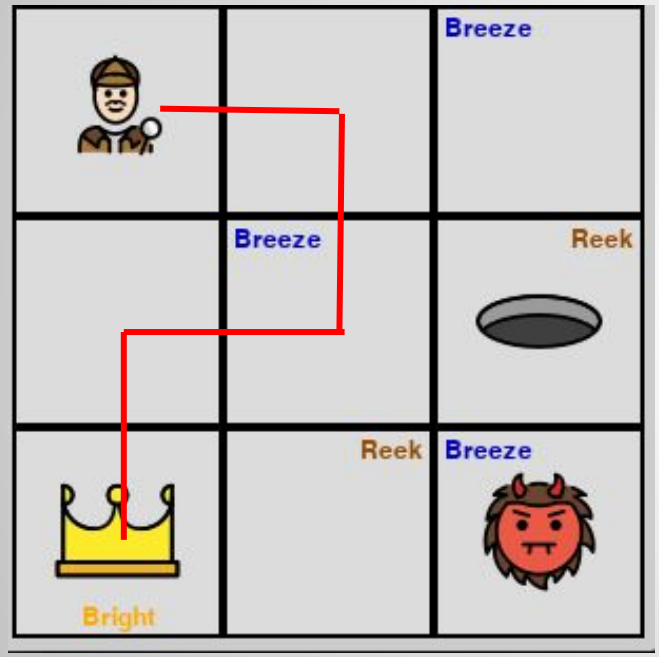
- **Reducción al Absurdo (entails)**
  - Evaluación mediante contradicción: *KB U {¬α}*
  - Forzar hipótesis de peligro

- **Optimización DPLL (DPLL.py)**
  - Puros: asignación sin alternativas
  - Unitarios: descarte por cláusula única

- **Toma de decisiones (move)**
  - DPLL = False ==> casilla 100% segura
  - Movimiento seguro, exploración o retroceso

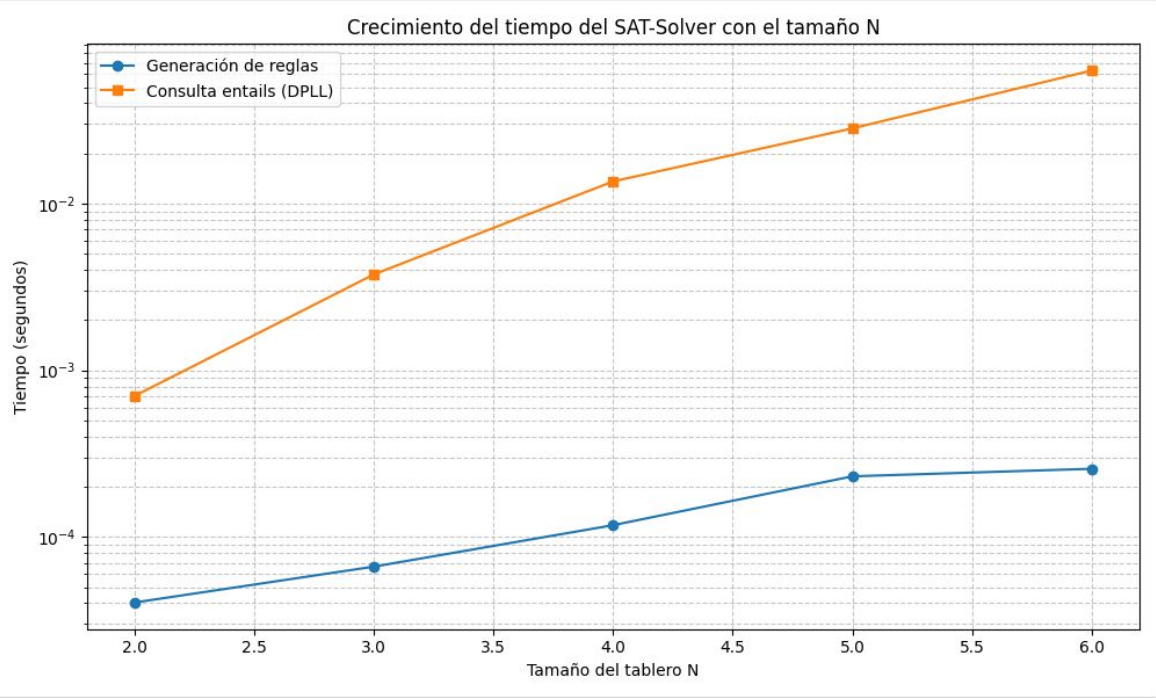


Resultados + conclusión



- Formalización mediante lógica proposicional
- Inferencia mediante DPLL
- Navegación autónoma en un entorno desconocido

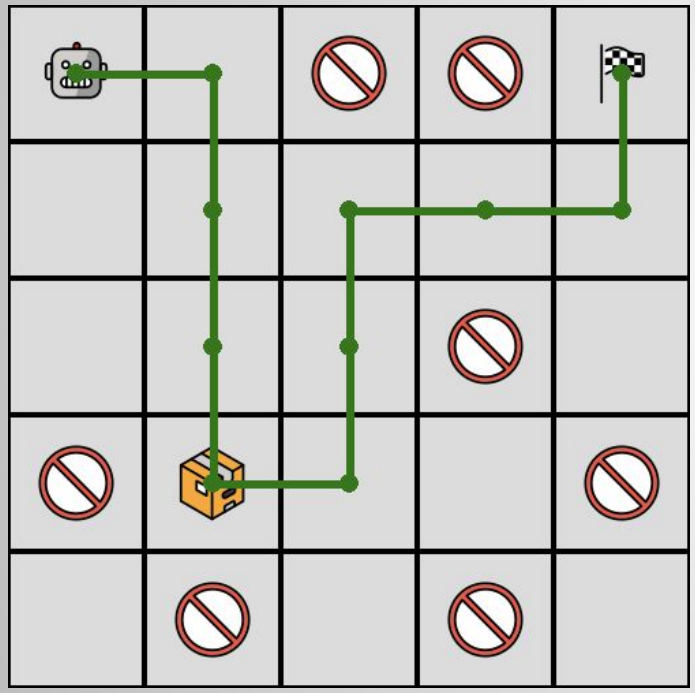
Experimentación



¿Tiempo polinómico?
N = range(2,7)

- Cláusulas Horn
- Crecimiento de variables O(n^2)
- Árbol prácticamente sin ramificar

Problema y formalización



1. **Problema**
   a. Robot de almacén: coger la caja y llevarla a la meta
   b. Obstáculos aleatorios; siempre existe un camino
   c. Entorno determinista y totalmente observable
2. **Cálculo de situaciones**
   a. Argumentos: En(Agente, x, s), En(C, x, s), Sosteniendo(C, s)
   b. Acciones: Ir(x, y), Coger(C), Soltar(C)
   c. Init: En(Agente, [1,1], S0) $\wedge$ En(C, caja, S0)
3. **Objetivo**
   a. $\exists$ seq En(C, meta, Resultado(seq, S0)) $\wedge$ $\neg$Sosteniendo(C, Resultado(seq, S0))

Axiomas del dominio

Precondición

Ir(x, y): En(Agente, x, s) ∧ Adyacente(x, y) ∧ ¬Obstáculo(y)

Coger(C): En(Agente, x, s) ∧ En(C, x, s) ∧ ¬Sosteniendo(C, s)

Soltar(C): En(Agente, x, s) ∧ Sosteniendo(C, s)

Efecto

Notación: s' = Resultado(a, s)

Ir(x, y): En(Agente, y, s') ∧ ¬En(Agente, x, s')

La caja sostenida no tiene En(C, ·): va con el robot

Coger(C): Sosteniendo(C, s') ∧ ¬En(C, x, s')

Soltar(C): ¬Sosteniendo(C, s') ∧ En(C, x, s')

Estado-sucesor

Aplicable(a, s) ⇒ (F(s') ⇔ EfectoPos(a, F) ∨ [F(s) ∧ ¬EfectoNeg(a, F)])

Resuelve el problema del marco: lo que no cambia se conserva

En el código: s' = (s − EfectoNeg) ∪ EfectoPos

Planificación basada en conocimiento: robot de reparto

Búsqueda hacia delante con BFS

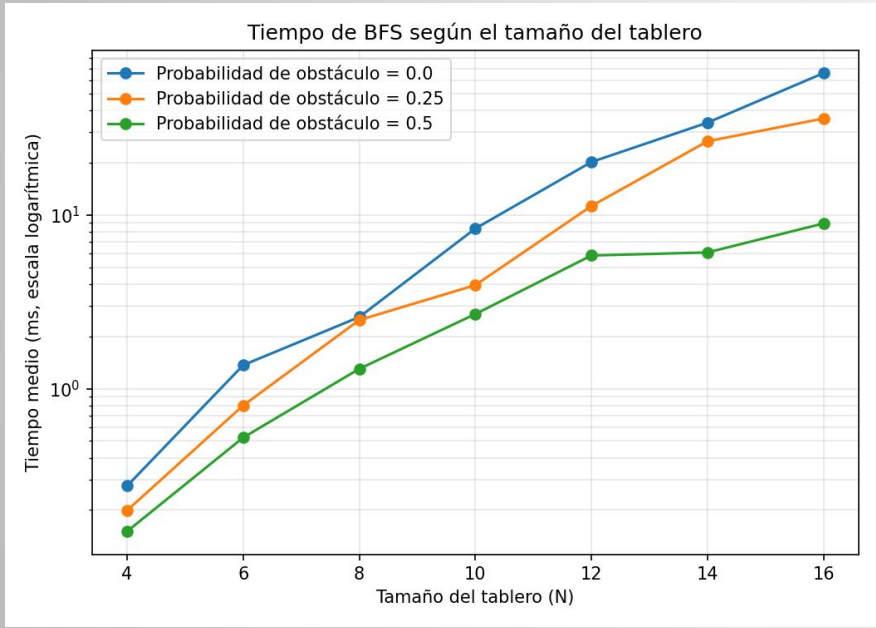
1. Frontera FIFO con (S0, [ ]): se avanza desde la situación inicial
2. Sacar la situación más antigua: primero planes de 1 acción, luego de 2...
3. Acciones aplicables según las precondiciones
4. Situación siguiente con el axioma estado-sucesor
5. Si ya se visitó, se descarta; si cumple el objetivo, se devuelve el plan

Ejemplo (N = 5, figura anterior)

1. Ir([1,1], [2,1])
2. Ir([2,1], [2,2])
3. Ir([2,2], [2,3])
4. Ir([2,3], [2,4])
5. Coger(C)
6. Ir([2,4], [3,4])
7. Ir([3,4], [3,3])
8. Ir([3,3], [3,2])
9. Ir([3,2], [4,2])
10. Ir([4,2], [5,2])
11. Ir([5,2], [5,1])
12. Soltar(C)

12 acciones

Resultados y conclusiones



1. **Experimento**
   a. Tamaño N de 4 a 16; probabilidad de obstáculo 0, 0.25 y 0.5
   b. 50 tableros por configuración; tiempo medio de BFS
2. **Conclusiones**
   a. El tiempo crece como N^4: ≈ ×25 al duplicar N (de 8 a 16)
   b. Más obstáculos, menos tiempo: ≈ 65 ms frente a ≈ 9 ms con N = 16
   c. Plan óptimo en menos de 0.1 s en todos los casos
3. **Limitación**
   a. Siempre existe solución: no se mide el peor caso